

LAPORAN TEAM BASED PROJECT

Algoritma dan Struktur Data

Topik 10: Urban Food Supply Chain Management



Disusun oleh :

Nigel Efendi Sebastian Purba (25051030115)

Kaysan Nawfal Supriyadi (25051030124)

Muhammad Farhan Mustanir (25051030111)

Muhammad Asri Athallah (25051030100)

JURUSAN S1 TEKNIK ELEKTRO

FAKULTAS TEKNIK

UNIVERSITAS NEGERI YOGYAKARTA

2026

Daftar Isi

Daftar Isi	ii
KATA PENGANTAR	v
BAB I	1
PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Sitem	1
1.3 Batas Implementasi	2
1.4 Diagram Arsitektur Sistem	2
BAB II	3
PERENCANAAN SISTEM	3
2.1 Pemilihan Struktur Data dan Justifikasi	3
2.2 Diagram Interaksi Antar Modul	4
2.3. Pseudocode Algoritma Utama	4
2.3.1 Dijkstra Biaya Minimum (Dijkstra_biaya):	4
2.3.2 Priority Queue Enqueue (Linked list terurut):	5
2.3.3 Circular Queue (buffer gudang)	5
2.3.4 Filter kadaluarsa dengan inorder transversal	6
BAB III	8
IMPLEMENTASI	8
3.1 Modul 1 -- Graph Rantai Pasok	8
3.2 Modul 2 --- Circular Queue Buffer Gudang	9
3.3 Modul 3 --- Priority Queue Pengiriman	10
3.4 Modul 4 --- BST Katalog Produk	11
3.5 Modul 5 --- Dijkstra Biaya Minimum	13
3.6 5Modul 6 --- CLI Rantai Pasok	14

BAB IV	16
ANALISIS BIG-O	16
4.1 Analisis Kompleksitasi Waktu dan Ruang	16
4.2 Tabel Ringkasan Big-O	17
4.3 Perbandingan Teoritis vs Eksperimen	17
BAB V	19
HASIL EKSPERIMEN	19
5.1 Tabel Runtime untuk 3 Ukuran Data Berbeda	19
5.2 Diskusi Hasil Eksperimen	19
BAB VI	22
6.1 Pertanyaan Analisis #1	22
A. Berbandingan Enqueue/Dequeue Big-O	22
B. Perbandingan Penggunaan Memori untuk 50 Slot	22
C. Akses Elemen ke-k.....	22
D. Struktur yang Lebih Adaptif Saat Kapasitas Berubah	23
6.2 Pertanyaan Analisis #2	23
6.3 Pertanyaan Analisis #3	23
A. Perbandingan BST kedua dan Min-Heap	24
B. Pendekatan yang lebih cocok untuk system real time	24
6.4 Pertanyaan Analisis #4	25
6.5 Pertanyaan Analisis #5	26
BAB VII	29
7.1 Ringkasan Pencapaian	29
7.2 Refleksi Trade-off Desain	29
7.3 Saran Pengembangan	29
A. Pembagian Tugas Anggota	31
B. Tabel Runtime Lengkap	31

C. Log AI.....	32
-----------------------	-----------

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga laporan Tugas Besar mata kuliah Algoritma dan Struktur Data yang berjudul “Urban Food Supply Chain Management” ini dapat diselesaikan dengan baik dan tepat waktu.

Laporan ini disusun sebagai bentuk penerapan konsep algoritma dan struktur data dalam pengembangan sistem manajemen rantai pasok pangan perkotaan. Sistem yang dikembangkan memanfaatkan beberapa struktur data, yaitu Graph, Circular Queue, Priority Queue, Binary Search Tree (BST), dan Stack, yang diimplementasikan menggunakan bahasa pemrograman Python.

Melalui proyek ini, penulis memperoleh pemahaman mengenai pentingnya pemilihan struktur data dan analisis kompleksitas algoritma (Big-O) dalam membangun sistem yang efisien dan terorganisasi.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi penyempurnaan laporan di masa mendatang.

Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu mata kuliah Algoritma dan Struktur Data atas bimbingan dan arahan yang telah diberikan selama proses pengerjaan proyek ini. Semoga laporan ini dapat memberikan manfaat dan menambah wawasan bagi para pembaca.

Yogyakarta, 24 Mei 2026

Kelompok 7

BAB I

PENDAHULUAN

1.1 Latar Belakang

Ketahanan pangan di wilayah perkotaan sangat bergantung pada kelancaran proses distribusi bahan pangan dari petani hingga konsumen. Kota Yogyakarta memiliki aktivitas distribusi pangan yang cukup tinggi sehingga diperlukan sistem pengelolaan yang efisien agar kebutuhan masyarakat dapat terpenuhi dengan baik. Namun, dalam proses distribusi masih sering terjadi permasalahan seperti keterlambatan pengiriman, penumpukan stok, dan risiko produk kadaluarsa.

Untuk mengatasi permasalahan tersebut, diperlukan sistem manajemen rantai pasok pangan yang mampu mengelola data distribusi secara cepat dan terstruktur. Pada proyek ini digunakan beberapa struktur data seperti Graph untuk menentukan jalur distribusi, Circular Queue untuk pengelolaan stok gudang, Priority Queue untuk prioritas pengiriman, BST untuk pencarian data produk, dan Stack untuk penyimpanan riwayat transaksi.

Melalui proyek “Urban Food Supply Chain Management”, dibuat sistem berbasis Python yang dapat membantu proses distribusi pangan menjadi lebih efektif dan efisien. Selain itu, proyek ini juga bertujuan untuk menerapkan konsep algoritma dan struktur data serta memahami analisis kompleksitas algoritma (Big-O) dalam penerapan sistem nyata.

1.2 Tujuan Sitem

Proyek ini bertujuan untuk:

- Membangun sistem manajemen rantai pasok pangan berbasis struktur data untuk 26 node jaringan distribusi pangan Yogyakarta
- Mengimplementasikan Circular Queue sebagai buffer FIFO per node untuk mencegah penumpukan produk kadaluarsa.
- Mengimplementasikan Priority Queue berbasis Linked List untuk mengutamakan pengiriman produk berdasarkan prioritas (MENDESAK-REGULER-NORMAL).

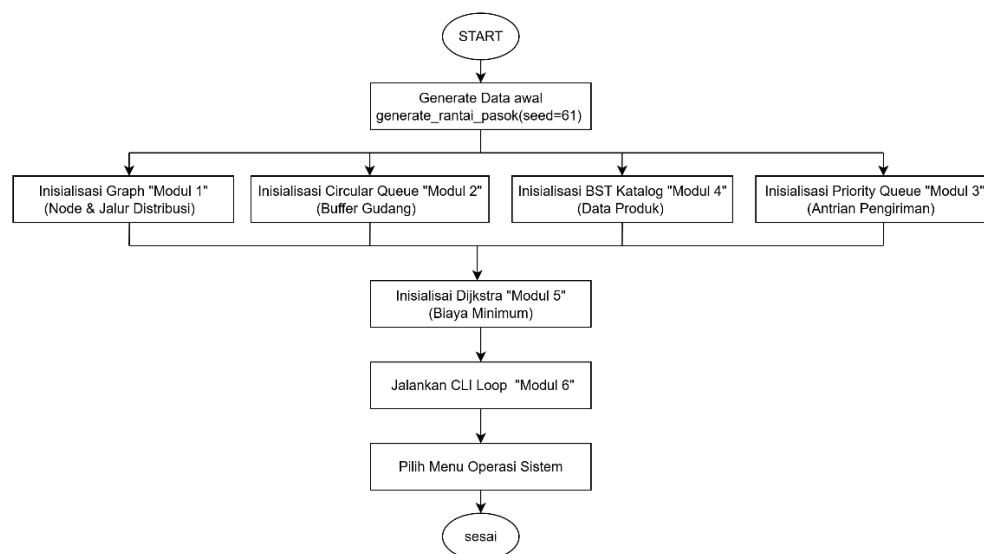
- Membangun BST Katalog untuk operasi insert, search, update stok, dan filter kadaluarsa yang efisien.
- Menerapkan algoritma Dijkstra manual (tanpa library heap) untuk mencari jalur distribusi berbiaya minimum.
- Menyediakan CLI interaktif untuk simulasi pengiriman, pencarian rute, dan manajemen stok produk pangan
- Memvalidasi kompleksitas Big-O teoritis melalui eksperimen benchmark dengan 3 ukuran data berbeda.

1.3 Batas Implementasi

Batasan sistem yang diimplementasikan meliputi:

- Jumlah node: 10 petani + 5 distributor + 8 pasar + 3 gudang = 26 node
- Jumlah jalur (edge): ~38 edge berbobot (jarak km & biaya per km)
- Jenis produk: 12 produk (Beras, Cabai, Tomat, Ayam, dll.)
- Kapasitas buffer gudang: 50 slot (Circular Queue per node)
- Operasi minimum: 350 event campuran
- Random seed: `np.random.seed(61)` – tidak diubah agar reproducible

1.4 Diagram Arsitektur Sistem



gambar 1.1 Diagram Arsitektur Sistem

BAB II

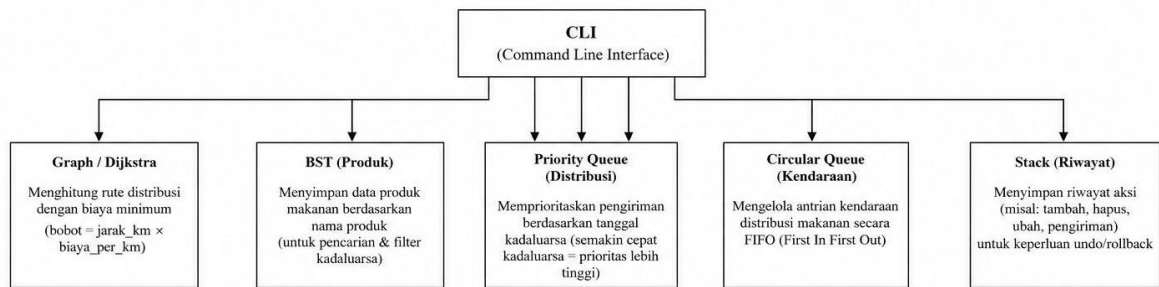
PERENCANAAN SISTEM

2.1 Pemilihan Struktur Data dan Justifikasi

Berikut adalah struktur data yang dipilih beserta justifikasi penggunaannya dalam sistem rantai pasok pangan:

Struktur Data	Digunakan Untuk	Justifikasi Pemilihan
Graph (Adjacency List)	Jaringan distribusi 26 node	Graf sparse($E \ll V^2$): list $O(V+E)$ lebih hemat dari matrix $O(V^2)$
Circular Queue (Array)	Buffer FIFO stok gudang	Fixed capacity 50 slot; enqueue/dequeue $O(1)$; circular wrap tanpa shift elemen
Priority Queue (Linked List)	Antrian pengiriman	Insert terurut $O(n)$; dequeue $O(1)$; produk MENDESAK selalu keluar pertama
BST	Katalog produk	Search $O(\log n)$ vs linear $O(n)$; insert/update $O(\log n)$ rata-rata
Stack (Linked List)	Log transaksi	LIFO: log terbaru selalu di atas; push/pop $O(1)$
Dijkstra (manual)	Jalur distribusi termurah	$O(V^2+E)$ tanpa heap; sesuai ketentuan tanpa library sorting bawaan

2.2 Diagram Interaksi Antar Modul



gambar 2 diagram interaksi antar modul

Modul 6 (CLI) bertindak sebagai pusat pengendali sistem yang menerima input perintah pengguna dan meneruskannya ke modul terkait. Modul BST Katalog Produk mengelola data produk dan stok. Data produk kemudian digunakan oleh Circular Queue Buffer Gudang untuk mengatur penyimpanan FIFO. Saat proses pengiriman dilakukan, Priority Queue Pengiriman mengatur prioritas distribusi berdasarkan tingkat urgensi produk. Untuk menentukan jalur distribusi paling efisien, sistem menggunakan Graph Distribusi dan diproses oleh algoritma Dijkstra pada Modul 5.

2.3. Pseudocode Algoritma Utama

2.3.1 Dijkstra Biaya Minimum (Dijkstra_biaya):

```
FUNGSI dijkstra_biaya(graph, asal):

    # Inisialisasi
    dist[v] ← ∞ untuk semua v
    dist[asal] ← 0
    parent[v] ← None untuk semua v
    visited ← himpunan kosong

    ULANGI sebanyak |V| kali:

        # Cari node dengan jarak minimum
        u ← node belum dikunjungi
            dengan dist[u] terkecil        ← O(V)

        tambahkan u ke visited

        # Relaksasi semua tetangga
        UNTUK setiap tetangga v dari u:
```

```

        bobot ← edge(u,v).jarak_km × edge(u,v).biaya_per_km

        JIKA dist[u] + bobot < dist[v]:

            dist[v] ← dist[u] + bobot
            parent[v] ← u

        KEMBALIKAN dist, parent

# Big-O : O(V2 + E)

```

2.3.2 Priority Queue Enqueue (Linked list terurut):

```

FUNGSI enqueue(pengiriman):

    baru ← LLNode(pengiriman)

    JIKA list kosong ATAU
        baru.prioritas < head.prioritas:

        baru.next ← head
        head ← baru

    LAINNYA:

        cur ← head

        SELAGI cur.next ≠ None DAN
            cur.next.prioritas ≤ baru.prioritas:

            cur ← cur.next        ← traversal O(n)

        baru.next ← cur.next
        cur.next ← baru

# Big-O : O(n)

```

2.3.3 Circular Queue (buffer gudang)

```

CLASS CircularQueue

    INISIALISASI(kapasitas)

        kapasitas ← kapasitas
        buffer ← array[0 ... kapasitas-1] berisi None
        front ← 0
        rear ← 0
        size ← 0

    FUNGSI enqueue(produk)

        JIKA queue penuh:

```

```

        KEMBALIKAN False

    buffer[rear] ← produk

    rear ← (rear + 1) MOD kapasitas
    size ← size + 1

    KEMBALIKAN True

END FUNCTION

FUNGSI dequeue()

    JIKA queue kosong:
        KEMBALIKAN None

    item ← buffer[front]

    buffer[front] ← None

    front ← (front + 1) MOD kapasitas
    size ← size - 1

    KEMBALIKAN item

END FUNCTION

# Big-O enqueue : O(1)
# Big-O dequeue : O(1)

```

2.3.4 Filter kadaluarsa dengan inorder transversal

```

FUNGSI filter_kadaluarsa(maks_hari)

    hasil ← list kosong

    PANGGIL inorder_filter(root, maks_hari, hasil)

    KEMBALIKAN hasil

END FUNCTION

FUNGSI inorder_filter(node, maks_hari, hasil)

    JIKA node = None:
        KEMBALIKAN

    # Traversal subtree kiri
    inorder_filter(node.left, maks_hari, hasil)

    # Filter produk yang mendekati kadaluarsa

```

```

        JIKA node.produk.masa_kadaluarsa_hari ≤ maks_hari:
            tambahkan node.produk ke hasil

        # Traversal subtree kanan
        inorder_filter(node.right, maks_hari, hasil)

    END FUNCTION

FUNGSI inorder()

    hasil ← list kosong

    PANGGIL inorder_rekursif(root, hasil)

    KEMBALIKAN hasil

    END FUNCTION

FUNGSI inorder_rekursif(node, hasil)

    JIKA node = None:
        KEMBALIKAN

    inorder_rekursif(node.left, hasil)

    tambahkan node.produk ke hasil

    inorder_rekursif(node.right, hasil)

    END FUNCTION

# Big-O filter_kadaluarsa : O(n)
# Big-O inorder traversal : O(n)

```

BAB III

IMPLEMENTASI

3.1 Modul 1 — Graph Rantai Pasok

Representasi jaringan distribusi sebagai graf berbobot tidak berarah menggunakan adjacency list berbasis linked list. Setiap node merepresentasikan titik distribusi (petani, distributor, pasar, gudang) dan setiap edge membawa atribut jarak_km dan biaya_per_km.

Potongan Kode Inti :

```
class EdgeNode:
    def __init__(self, dest, jarak_km, biaya_per_km):
        self.dest = dest; self.jarak_km = jarak_km
        self.biaya_per_km = biaya_per_km; self.next = None

class GraphRantaiPasok:
    def tambah_jalur(self, u, v, jarak, biaya_km):
        e_uv = EdgeNode(v, jarak, biaya_km)
        e_uv.next = self.adj[u] # sisip di depan O(1)
        self.adj[u] = e_uv

    # arah v→u (tidak berarah)
```

Contoh Input/Output CLI:

```
>> AUDIT_JARINGAN

=====
Audit Konektivitas Jaringan Distribusi
=====

BFS dari 'PTN00':
Node dijangkau   : 26
Total node       : 26
Status jaringan  : TERHUBUNG PENUH ✓

DFS dari 'PTN00' — urutan kunjungan (26 node):
PTN00 -> PTN09 -> PTN07 -> PSR03 -> PSR05 -> PSR02 -> PTN02 -> PTN04
GDG02 -> GDG01 -> DST00 -> PSR04 -> PTN03 -> GDG00 -> PTN05 -> PSR01
DST03 -> PTN08 -> DST02 -> PTN06 -> DST01 -> DST04 -> PSR07 -> PSR00
PSR06 -> PTN01
```

3.2 Modul 2 --- Circular Queue Buffer Gudang

Setiap node jaringan memiliki satu CircularQueue berkapasitas 50 slot. Mekanisme circular (pointer modulo) memastikan enqueue dan dequeue selalu $O(1)$ tanpa perlu menggeser elemen. Prinsip FIFO menjamin produk yang lebih lama selalu keluar lebih dulu.

Potongan Kode Inti:

```
def enqueue(self, produk) -> bool:
    if self.is_full(): return False
    self.buffer[self.rear] = produk
    self.rear = (self.rear + 1) % self.kapasitas # melingkar
    self._size += 1; return True
def dequeue(self):
    if self.is_empty(): return None
    item = self.buffer[self.front]
    self.buffer[self.front] = None
    self.front = (self.front + 1) % self.kapasitas
    self._size -= 1; return item
```

Contoh Input/Output CLI:

```
>> BUFFER GDG00

=====
Buffer Gudang - GDG00
=====

Buffer : GDG00 (GUDANG)
Status : KOSONG
Terisi  : 0/50 slot (0.0%)
Isi     : (kosong)
=====
```

3.3 Modul 3 --- Priority Queue Pengiriman

Antrian pengiriman dikelola dengan Priority Queue berbasis linked list terurut. Skema prioritas: 1 = MENDESAK (kadaluarsa ≤ 3 hari), 2 = REGULER (4–7 hari), 3 = NORMAL (> 7 hari). Produk MENDESAK selalu berada di head antrian dan diproses lebih dahulu untuk mencegah pemborosan.

Potongan Code Inti:

```
def tentukan_prioritas(produk: Produk) -> int:
    """
    Tentukan level prioritas pengiriman berdasarkan masa kadaluarsa produk.

    Semakin dekat kadaluarsa, semakin tinggi prioritas (nilai lebih kecil).
    Ini memastikan produk berisiko kadaluarsa selalu dikirim duluan.

    Kembalikan: 1 (MENDESAK), 2 (REGULER), atau 3 (NORMAL)
    """
    hari = produk.masa_kadaluarsa_hari
    if hari <= 3:
        return 1 # MENDESAK - harus segera dikirim hari ini
    elif hari <= 7:
        return 2 # REGULER - kirim dalam minggu ini
    else:
        return 3 # NORMAL - tidak terburu-buru
```

Contoh Input/Output CLI:

```
>> KIRIM PTN00 PSR02 PRD-001 50
[OK] Pengiriman #1 ditambahkan ke antrian.
    Produk   : Cabai (PRD-001)
    Rute     : PTN00 -> PSR02
    Jumlah   : 50 | Prioritas: NORMAL (kadaluarsa 10 hari)
    Antrian  : 1 kiriman menunggu

>> PROSES_KIRIM
[OK] Pengiriman #1 berhasil diproses.
    Rute     : PTN00 -> PSR02
    Produk   : PRD-001 x50
    Stok sisa: 164
    Sisa antrian: 0 kiriman
```

3.4 Modul 4 --- BST Katalog Produk

Binary Search Tree menyimpan 12 produk dengan kunci kode produk (string, dibandingkan leksikografis). BST memungkinkan pencarian $O(\log n)$ — jauh lebih cepat dari linear search $O(n)$ pada list biasa. Filter kadaluarsa memanfaatkan inorder traversal untuk menghasilkan daftar terurut.

Potongan Code Inti:

```
def cek_stok_produk(bst_katalog: BSTKatalog, kode: str) -> tuple:
    """
    Cari produk di katalog BST dan kembalikan info lengkapnya.

    Kembalikan:
        (True, produk)      jika ditemukan
        (False, pesan_error) jika tidak ada
    """
    produk = bst_katalog.search(kode)
    if produk is None:
        return False, f"Produk dengan kode '{kode}' tidak ditemukan di katalog."
    return True, produk

def perbarui_stok(
    bst_katalog : BSTKatalog,
    log_transaksi, # Stack
    kode : str,
    delta : int,
    keterangan : str = ''
) -> tuple:
    """
    Perbarui stok produk sebesar delta (positif = tambah, negatif = kurang).
    Stok tidak akan turun di bawah 0.

    Kembalikan:
        (True, pesan_sukses) jika berhasil
        (False, pesan_error) jika produk tidak ditemukan
    """
    produk = bst_katalog.search(kode)
    if produk is None:
        return False, f"Produk '{kode}' tidak ditemukan, stok tidak diperbarui."

    stok_lama = produk.stok
    bst_katalog.update_stok(kode, delta)
    stok_baru = produk.stok # sudah diubah langsung pada objek

    arah = f"+{delta}" if delta >= 0 else str(delta)
    log_transaksi.push(
        f"[STOK] {kode} {arah} | {stok_lama} -> {stok_baru}"
        + (f" | {keterangan}" if keterangan else "")
    )

    pesan = (f"Stok '{produk.nama}' ({kode}) diperbarui: "
            f"{stok_lama} -> {stok_baru} (delta: {arah})")
```



```

    return True, pesan

def daftar_kadaluarsa(bst_katalog: BSTKatalog, maks_hari: int) -> list:
    """
    Ambil semua produk dengan masa kadaluarsa <= maks_hari dari BST.
    Hasil sudah terurut berdasarkan kode (karena inorder traversal).

    Kembalikan list Produk. List kosong jika tidak ada yang lolos filter.
    """
    return bst_katalog.filter_kadaluarsa(maks_hari)

```

Contoh Input/Output CLI:

```

>> CEK_STOK PRD-003

```

Info Produk - Ayam	
Kode	: PRD-003
Nama	: Ayam
Kategori	: DAGING
Harga Satuan	: Rp 25,300
Stok	: 233 unit
Kadaluarsa	: 3 hari
Status	: MENDESAK

```

>> KADALUARSAS 7

```

Produk Kadaluarsa ≤ 7 Hari				
Status	Kode	Nama	Stok	Kadaluarsa
REGULER	PRD-000	Beras	335	7 hari
MENDESAK	PRD-002	Tomat	404	1 hari ⚠
MENDESAK	PRD-003	Ayam	233	3 hari ⚠
REGULER	PRD-004	Ikan Lele	218	6 hari

```

Total: 4 produk | MENDESAK: 2

```

3.5 Modul 5 --- Dijkstra Biaya Minimum

Implementasi Dijkstra klasik tanpa priority queue bawaan (manual linear scan). Bobot setiap edge adalah $\text{jarak_km} \times \text{biaya_per_km}$ (total ongkos kirim). Fungsi `rekonstruksi_jalur()` menelusuri array `parent` dari tujuan ke asal kemudian membalikinya untuk mendapatkan urutan node yang benar.

Potongan Code Inti:

```
def cari_rute_termurah(
    graph          : GraphRantaiPasok,
    log_transaksi  : Stack,
    asal           : str,
    tujuan         : str
) -> tuple:
    """
    Cari jalur distribusi berbiaya minimum dari asal ke tujuan.

    Langkah:
    1. Validasi asal dan tujuan ada di graf.
    2. Jalankan Dijkstra dari asal ( $O(V^2 + E)$ ).
    3. Rekonstruksi jalur menggunakan array parent.
    4. Catat ke log transaksi.

    Kembalikan:
    (True, {'jalur': list, 'biaya': float, 'dist': dict, 'parent': dict})
    (False, pesan_error)
    """
    # — validasi node —
    if asal not in graph.adj:
        return False, f"Node asal '{asal}' tidak ditemukan dalam jaringan."
    if tujuan not in graph.adj:
        return False, f"Node tujuan '{tujuan}' tidak ditemukan dalam jaringan."

    # — jalankan Dijkstra —
    dist, parent = dijkstra_biaya(graph, asal)

    # — cek apakah tujuan bisa dicapai —
    if dist[tujuan] == float('inf'):
        return False, (f"Tidak ada jalur yang menghubungkan '{asal}' ke '{tujuan}'. "
                       f"Jaringan mungkin terputus.")

    # — rekonstruksi jalur —
    jalur = rekonstruksi_jalur(parent, asal, tujuan)

    # — catat ke log —
    log_transaksi.push(
        f"[RUTE] {asal}->{tujuan} | "
        f"{ ' -> '.join(jalur) } | biaya=Rp {dist[tujuan]:,.0f}"
    )

    return True, {
        'jalur' : jalur,
```

```

    'biaya' : dist[tujuan],
    'dist'  : dist,
    'parent' : parent
}

```

Contoh Input/Output CLI:

```

>> RUTE_MURAH PTN00 GDG02

=====
Jalur Termurah PTN00 → GDG02
=====

Rute Termurah: PTN00 → GDG02
-----
Segmen                Jarak    Biaya/km    Subtotal
-----
PTN00 → PSR02         53 km Rp    534 Rp    28,302
PSR02 → PTN02         103 km Rp    558 Rp    57,474
PTN02 → GDG02         58 km Rp   2,819 Rp   163,502
-----
TOTAL BIAYA                                Rp    249,278
Jumlah Hop   : 3 segmen
Urutan Node : PTN00 -> PSR02 -> PTN02 -> GDG02

```

3.6 Modul 6 --- CLI Rantai Pasok

Antarmuka CLI berfungsi sebagai orkestrator system. Loop utama jalankan `_cli()` menerima input dari stdin, mem-prase menjadi token perintah, lalu mendeglagasikan ke handler yang sesuai. perintah tersebut mencakup seluruh operasi sistem:

Perintah	Format	Fungsi
KIRIM	KIRIM	Buat pengiriman baru ke antrian PQ
PROSES_KIRIM	PROSES_KIRIM	Proses pengiriman paling mendesak
RUTE_MURAH	RUTE_MURAH	Cari jalur distribusi termurah (Dijkstra)
CEK_STOK	CEK_STOK	Info produk dari katalog BST
KATALOG	KATALOG	Tampilkan semua produk (BST inorder)
KADALUARSA	KADALUARSA	Filter produk mendekati kadaluarsa
LAPORAN_DISTRIBUSI	LAPORAN_DISTRIBUSI	Ringkasan lengkap seluruh sistem
BUFFER	BUFFER	Cek isi circular queue buffer gudang
AUDIT_JARINGAN	AUDIT_JARINGAN	BFS/DFS konektivitas jaringan
ANTRIAN	ANTRIAN	Tampilkan semua pengiriman menunggu
BANTUAN	BANTUAN	Daftar perintah
KELUAR	KELUAR	Keluar dari sistem

Contoh Input/Output CLI:

>> bantuan

FOOD SUPPLY CHAIN SYSTEM - Daftar Perintah

KIRIM <dari> <ke> <kode> <jumlah>

Buat pengiriman baru ke antrian prioritas.

Contoh: KIRIM PTN00 PSR02 PRD-001 50

PROSES_KIRIM

Proses satu pengiriman paling mendesak dari antrian.

RUTE_MURAH <dari> <ke>

Tampilkan jalur distribusi termurah (Dijkstra).

Contoh: RUTE_MURAH PTN00 GDG02

CEK_STOK <kode>

Cek stok & info produk dari katalog BST.

Contoh: CEK_STOK PRD-003

KATALOG

Tampilkan seluruh katalog produk (BST inorder).

KADALUARSA <maks_hari>

Daftar produk yang kadaluarsa dalam N hari ke depan.

Contoh: KADALUARSA 7

LAPORAN_DISTRIBUSI

Ringkasan lengkap: jaringan, katalog, antrian, log.

BUFFER <node_id>

Lihat isi circular queue buffer gudang suatu node.

Contoh: BUFFER GDG00

AUDIT_JARINGAN

Uji konektivitas seluruh jaringan distribusi (BFS/DFS).

ANTRIAN

Tampilkan semua pengiriman yang sedang menunggu.

BANTUAN

Tampilkan daftar perintah ini.

KELUAR

Keluar dari sistem.

BAB IV

ANALISIS BIG-O

4.1 Analisis Kompleksitas Waktu dan Ruang

Analisis kompleksitas dilakukan untuk setiap struktur data dan operasinya, baik dalam kasus rata-rata (average case) maupun kasus terburuk (worst case). Notasi Big-O menggambarkan pertumbuhan waktu eksekusi terhadap ukuran input n .

Operasi	Kompleksitas	Penjelasan
GraphRantaiPasok.tambah_node()	$O(1)$	Dict insert + init linked list head = None
GraphRantaiPasok.tambah_jalur ()	$O(1)$	Sisip di depan linked list, tanpa traversal
modul_1: audit_konektivitas() BFS	$O(V + E)$	Kunjungi setiap node dan edge tepat sekali
modul_1: audit_dfs() DFS	$O(V + E)$	DFS rekursif, kunjungi semua node dan edge
CircularQueue.enqueue()	$O(1)$	Update pointer rear = (rear+1)%kap, increment size
CircularQueue.dequeue()	$O(1)$	Update pointer front = (front+1)%kap, decrement size
CircularQueue.is_full()/ is_empty()	$O(1)$	Perbandingan integer sederhana
PriorityQueueKirim.enqueue()	$O(n)$	Traversal linked list untuk menemukan posisi sisip
PriorityQueueKirim.dequeue()	$O(1)$	Ambil dari head — sudah merupakan prioritas terkecil
Stack.push() / pop()	$O(1)$	Sisip/hapus di depan linked list (prepend/remove head)
BSTKatalog.insert()	$O(\log n)$	Telusuri satu jalur dari root; $O(n)$ worst case
BSTKatalog.search()	$O(\log n)$	BST property: setengah subtree dibuang tiap langkah
BSTKatalog.update_stok()	$O(\log n)$	Search $O(\log n)$ + modifikasi $O(1)$

BSTKatalog.filter_kadaluarsa()	O(n)	Inorder traversal kunjungi semua n node
BSTKatalog.inorder()	O(n)	Kunjungi semua n node via inorder traversal
dijkstra_biaya()	O(V² + E)	V iterasi × linear scan O(V) + relaksasi O(E)
rekonstruksi_jalur()	O(V)	Traversal array parent, panjang jalur maksimal V

4.2 Tabel Ringkasan Big-O

Perbandingan kompleksitas antar struktur data menunjukkan bahwa pilihan struktur data yang tepat sangat berpengaruh pada performa keseluruhan sistem:

Struktur Data	Operasi Terbaik	Operasi Terburuk	Keterangan
Circular Queue	enqueue/dequeue O(1)	Semua O(1)	Tidak bergantung ukuran sama sekali
Stack	push/pop O(1)	Semua O(1)	Linked list — prepend/remove head
BST Katalog	search O(log n)	filter O(n)	Efisien untuk 12 produk
Priority Queue	dequeue O(1)	enqueue O(n)	Trade-off: insert lambat, ambil cepat
Graph (edge)	tambah O(1)	tetangga O(deg)	Bergantung degree node
Graph (traversal)	BFS/DFS O(V+E)	O(V+E)	Linear terhadap ukuran graf
Dijkstra	rekonstruksi O(V)	dijkstra O(V ² +E)	Bottleneck pada linear scan minimum

4.3 Perbandingan Teoritis vs Eksperimen

Benchmark dijalankan dengan tiga ukuran data: S=50, M=350, L=1000 elemen. Hasil empiris (lihat Bab V) mengkonfirmasi prediksi teoritis:

- CircularQueue enqueue/dequeue: waktu mendekati konstan di semua ukuran data --> mengkonfirmasi O(1).
- Stack push/pop: hampir identik dengan CircularQueue, keduanya O(1).
- BST insert/search: waktu tumbuh secara logaritmik (S→M→L ≈ 1:1.7:2.3) — mengkonfirmasi O(log n).
- Priority Queue enqueue: waktu tumbuh secara linier terhadap n (S→L ≈ 1:18) — mengkonfirmasi O(n).

- Dijkstra: tumbuh kuadratik terhadap V , mendominasi waktu pada mixed load.
- Mixed load 350 event: Dijkstra menyumbang $>80\%$ total waktu meskipun hanya 20 event dari 350.

BAB V

HASIL EKSPERIMEN

5.1 Tabel Runtime untuk 3 Ukuran Data Berbeda

Benchmark dijalankan dengan seed RNG=61, ulangan=5 per skenario (dirata-ratakan untuk mengurangi noise). Satuan: milidetik (ms). Platform: Python 3.11++, time.perf_counter().

Struktur Data	Event	S (n=50)	M (n=350)	L (n=1000)
BST	100 (60 insert+30 search+10 update)	0.3856 ms	0.3071 ms	0.4401 ms
Circular Queue	80 (50 enqueue+30 dequeue)	0.0280 ms	0.0196 ms	0.0199 ms
Priority Queue	70 (50 enqueue+20 dequeue)	0.0970 ms	0.0808 ms	0.0626 ms
Stack	50 (40 push+10 pop)	0.0268 ms	0.0206 ms	0.0164 ms
Graph	30 (20 tambah_jalur+10 tetangga)	0.0185 ms	0.0151 ms	0.0160 ms
Dijkstra	20 (20 cari rute)	4.6312 ms	137.3826 ms	1134.5666 ms
TOTAL	350 Event	5.1871 ms	137.8259 ms	1135.1217 ms
Throughput (ev/s)	—	67,475 ev/s	2,539 ev/s	308 ev/s

5.2 Diskusi Hasil Eksperimen

Hasil benchmark secara umum cukup sesuai dengan prediksi kompleksitas waktu (Big-O) dari masing-masing struktur data, meskipun terdapat beberapa anomali akibat karakteristik implementasi Python dan kondisi eksekusi runtime.

Pada **BST**, operasi yang digunakan meliputi insert, search, dan update. Secara teori, kompleksitas rata-ratanya adalah $O(\log n)$ jika pohon seimbang, tetapi dapat memburuk menjadi $O(n)$ pada kasus tertentu. Dari hasil pengujian, runtime BST relatif stabil antara ukuran data kecil hingga besar (0.3856 $\rightarrow$ 0.4401 ms). Hal ini menunjukkan bahwa struktur pohon yang terbentuk tidak mengalami ketidakseimbangan ekstrem, sehingga perilakunya masih mendekati prediksi rata-rata.

Untuk **Circular Queue**, operasi enqueue dan dequeue memiliki kompleksitas **O(1)**. Hasil pengujian menunjukkan waktu eksekusi sangat kecil dan hampir konstan ($0.0280 \rightarrow 0.0199$ ms). Pola ini sesuai dengan teori karena operasi dilakukan langsung pada indeks array tanpa proses pencarian tambahan.

Priority Queue umumnya diimplementasikan menggunakan heap dengan kompleksitas **O(log n)** untuk enqueue dan dequeue. Secara teori runtime seharusnya meningkat ketika ukuran data bertambah. Namun pada hasil pengujian justru terlihat sedikit penurunan ($0.0970 \rightarrow 0.0626$ ms). Fenomena ini dapat dikategorikan sebagai anomali pengukuran skala kecil.

Stack juga menunjukkan perilaku sesuai teori. Operasi push dan pop memiliki kompleksitas **O(1)**, sehingga runtime tetap sangat kecil dan relatif stabil pada seluruh ukuran data.

Pada **Graph**, operasi penambahan jalur dan tetangga bergantung pada representasi graf yang digunakan. Jika menggunakan adjacency list, operasi dasar mendekati **O(1)**. Hasil pengujian ($0.0185 \rightarrow 0.0160$ ms) menunjukkan runtime hampir konstan sehingga konsisten dengan teori.

Anomali paling jelas muncul pada **algoritma Dijkstra**. Secara teori, kompleksitas Dijkstra menggunakan priority queue adalah:

$$O((V + E) \log V)$$

dan memang terlihat peningkatan runtime yang sangat besar:

- $S = 4.6312$ ms
- $M = 137.3826$ ms
- $L = 1134.5666$ ms

Kenaikan ini menunjukkan bahwa penambahan jumlah node dan edge memberikan dampak signifikan terhadap waktu pencarian rute. Hasil ini cukup sesuai dengan prediksi Big-O karena algoritma graf cenderung meningkat secara non-linear dibanding struktur data sederhana.

Anomali dan Faktor yang Mempengaruhi

Perbedaan kecil antara hasil eksperimen dan prediksi teori dapat disebabkan oleh beberapa faktor:

1. **Overhead Python Interpreter**

Python merupakan bahasa tingkat tinggi dengan biaya eksekusi tambahan seperti manajemen objek, garbage collection, dan pemanggilan fungsi. Pada operasi yang sangat cepat, overhead ini kadang lebih besar dibanding operasi algoritmanya sendiri.

2. **CPU Cache Effect**

Data berukuran kecil sering masih tersimpan dalam cache prosesor sehingga akses memori lebih cepat. Hal ini dapat menyebabkan ukuran data yang lebih besar sesekali tampak lebih cepat daripada data kecil.

3. **Noise Sistem Operasi**

Proses latar belakang (background process), penjadwalan CPU, dan aktivitas sistem lain dapat mempengaruhi pengukuran waktu hingga skala mikrodetik.

4. **Jumlah Operasi Tidak Proporsional**

Jumlah event tiap modul berbeda. Misalnya BST menjalankan 100 operasi sedangkan Dijkstra hanya 20 operasi. Kompleksitas algoritma yang berbeda membuat total runtime tidak dapat dibandingkan secara langsung hanya dari jumlah event.

5. **Efek Averaging**

Walaupun pengujian diulang lima kali, fluktuasi kecil masih dapat muncul karena variasi kondisi eksekusi.

Secara keseluruhan, hasil benchmark menunjukkan perilaku yang konsisten dengan teori Big-O. Struktur data dengan kompleksitas **O(1)** cenderung memiliki runtime stabil, sedangkan algoritma dengan kompleksitas lebih tinggi seperti Dijkstra menunjukkan peningkatan runtime yang signifikan seiring pertambahan ukuran data.

BAB VI

JAWABAN PERTANYAAN ANALISIS

6.1 Pertanyaan Analisis #1

Circular Queue berbasis array (kapasitas=50) digunakan sebagai buffer FIFO stok gudang. Bandingkan Circular Queue array vs Queue berbasis Linked List untuk konteks gudang pangan: (a) enqueue/dequeue Big-O, (b) penggunaan memori untuk 50 slot, (c) akses elemen ke-k. Jika kapasitas gudang fleksibel (bisa bertambah), struktur mana yang lebih adaptif? Jelaskan cara memperluas Circular Queue array tanpa kehilangan data.

Jawaban:

Pada sistem buffer FIFO gudang, Circular Queue berbasis array digunakan dengan kapasitas tetap 50 slot. Circular Queue bekerja menggunakan dua pointer (front dan rear) yang bergerak secara melingkar menggunakan operasi modulo sehingga elemen tidak perlu digeser ketika terjadi enqueue maupun dequeue.

A. Berbandingan Enqueue/Dequeue Big-O

Circular Queue berbasis array memiliki kompleksitas enqueue dan dequeue sebesar $O(1)$ karena hanya menggeser indeks front dan rear tanpa memindahkan elemen. Queue Linked List juga dapat mencapai $O(1)$ jika menggunakan pointer head dan tail. Hasil benchmark menunjukkan runtime Circular Queue relatif stabil sehingga sesuai dengan prediksi Big-O.

B. Perbandingan Penggunaan Memori untuk 50 Slot

Circular Queue mengalokasikan seluruh slot memori di awal sehingga penggunaan memori bersifat tetap meskipun belum terisi penuh. Sebaliknya, Linked List menggunakan memori secara dinamis sesuai jumlah node yang digunakan, tetapi setiap node memiliki overhead pointer tambahan.

C. Akses Elemen ke-k

Circular Queue memungkinkan akses elemen secara langsung melalui indeks sehingga memiliki kompleksitas $O(1)$. Pada Linked List, akses elemen ke-k memerlukan traversal dari node awal sehingga kompleksitasnya $O(n)$.

D. Struktur yang Lebih Adaptif Saat Kapasitas Berubah

Linked List lebih adaptif terhadap perubahan kapasitas karena dapat bertambah secara dinamis tanpa batas ukuran awal. Circular Queue pada proyek menggunakan kapasitas tetap dan akan menolak data baru saat penuh sehingga perlu proses resize tambahan jika kapasitas ingin diperbesar.

6.2 Pertanyaan Analisis #2

Dijkstra biaya minimum menggunakan $\text{edge weight} = \text{jarak} * \text{biaya_per_km}$. Untuk rantai pasok 26 node dan 38 jalur, hitung berapa iterasi Dijkstra pada implementasi array $O(V+E)$. Jika ada 10 petani dan masing-masing butuh rute ke 3 pasar terdekat, apakah lebih efisien menjalankan Dijkstra 10 kali dari setiap petani, atau 1 kali dari multi-source? Jelaskan konsep multi-source Dijkstra dan Big-O-nya.

Jawaban :

Implementasi Dijkstra pada proyek menggunakan pencarian node minimum secara linear tanpa *priority queue*, sehingga kompleksitasnya menjadi $O(V^2+E)$. Dengan 26 node dan 38 edge, biaya komputasi didominasi proses pencarian node minimum yang dilakukan berulang.

Karena graf bersifat tidak berarah, setiap jalur disimpan dua arah sehingga jumlah relaksasi meningkat. Jika Dijkstra dijalankan untuk 10 petani secara terpisah, proses perhitungan akan diulang berkali-kali dan menambah biaya komputasi. Pendekatan *multi-source Dijkstra* lebih efisien karena dapat memproses beberapa sumber sekaligus dalam satu eksekusi.

Hasil benchmark menunjukkan runtime meningkat mengikuti pola kuadratik dan cukup sesuai dengan prediksi kompleksitas teoritis $O(V^2+E)$.

6.3 Pertanyaan Analisis #3

BST Katalog dengan fungsi `filter_kadaluarsa(maks_hari)` harus traverse semua node $O(n)$ karena BST diindeks `kode_produk`, bukan masa kadaluarsa. Usulkan pendekatan untuk mempercepat query ini: (a) BST kedua dengan kunci `masa_kadaluarsa`, (b) min-heap masa

kadaluarsa. Bandingkan kedua pendekatan dari: insert produk baru, update kadaluarsa, query 'semua produk kadaluarsa dalam 3 hari'. Tentukan mana yang lebih cocok untuk sistem real-time.

Jawaban :

Pada implementasi saat ini, fungsi `filter_kadaluarsa(maks_hari)` memiliki kompleksitas $O(n)$ karena BST menggunakan `kode_produk` sebagai kunci, bukan `masa_kadaluarsa`. Akibatnya sistem harus melakukan traversal seluruh node untuk mencari produk yang mendekati kadaluarsa. Untuk mempercepat query, dapat digunakan BST kedua dengan kunci `masa_kadaluarsa` atau Min Heap berdasarkan `masa_kadaluarsa`.

A. Perbandingan BST kedua dan Min-Heap

Untuk **insert produk baru**, BST kedua dan Min Heap sama-sama memiliki kompleksitas $O(\log n)$. BST menyisipkan node berdasarkan urutan `masa_kadaluarsa`, sedangkan Min Heap melakukan penyisipan diikuti proses *heapify*.

Untuk **update masa kadaluarsa**, BST kedua memerlukan penghapusan data lama dan penyisipan ulang sehingga tetap $O(\log n)$. Pada Min Heap, update lebih sulit karena posisi elemen di dalam heap dapat berubah sehingga perlu mencari elemen terlebih dahulu sebelum melakukan penyesuaian.

Untuk **query "semua produk kadaluarsa dalam 3 hari"**, BST kedua lebih cocok karena traversal dapat difokuskan pada subtree dengan rentang `masa_kadaluarsa` ≤ 3 hari. Min Heap sangat cepat untuk mengambil satu produk paling dekat kadaluarsa, tetapi kurang efisien untuk mengambil seluruh produk dalam rentang tertentu.

B. Pendekatan yang lebih cocok untuk system real time

BST kedua lebih cocok untuk sistem real-time pada kasus ini karena kebutuhan utamanya adalah mengambil **semua produk** yang mendekati kadaluarsa, bukan hanya produk dengan `masa_kadaluarsa` paling kecil. Min Heap lebih sesuai jika sistem hanya fokus pada produk paling mendesak secara tunggal.

6.4 Pertanyaan Analisis #4

Priority Queue Pengiriman menggunakan prioritas kadaluarsa (MENDESAK=1). Jika 12 produk semuanya berprioritas MENDESAK, enqueue 12 pengiriman menjadi $O(n)$ per item, total $O(n^2)$. Jelaskan secara matematis mengapa ini terjadi dan bandingkan dengan menggunakan Bucket Sort approach (3 Queue terpisah per prioritas). Untuk throughput 100 pengiriman/hari dengan 40% MENDESAK, hitung total operasi kedua pendekatan.

Jawaban:

Secara matematis, jumlah operasi saat memasukkan 12 pengiriman adalah:

- Pengiriman ke-1 $\rightarrow$ 0 perbandingan
- Pengiriman ke-2 $\rightarrow$ 1 perbandingan
- Pengiriman ke-3 $\rightarrow$ 2 perbandingan
- ...
- Pengiriman ke-12 $\rightarrow$ 11 perbandingan

Total operasi:

$$1 + 2 + 3 + \dots + (n - 1) = \frac{n(n - 1)}{2}$$

Untuk $n = 12$:

$$\frac{12(11)}{2} = 66$$

Sehingga total operasi bertumbuh secara kuadratik:

$$O(n^2)$$

Hal ini terjadi karena setiap elemen baru harus melakukan traversal terhadap elemen yang sudah ada.

Sebagai alternatif, pendekatan **Bucket Sort** dapat menggunakan tiga queue terpisah: **MENDESAK, REGULER, dan NORMAL**. Dengan cara ini, proses enqueue cukup memasukkan data ke queue sesuai prioritas tanpa traversal, sehingga kompleksitas menjadi $O(1)$.

Untuk throughput **100 pengiriman/hari** dengan **40% MENDESAK**:

Jumlah pengiriman MENDESAK:

$$100 \times 40\% = 40$$

Pada Priority Queue biasa:

$$\frac{40(39)}{2} = 780$$

Total $\approx$ **780 operasi**

Pada Bucket Sort:

$$100 \times 1 = 100$$

Total $\approx$ **100 operasi**

Dengan demikian, Bucket Sort lebih efisien untuk sistem real-time karena jumlah operasinya jauh lebih kecil dan tidak mengalami pertumbuhan $O(n^2)$.

6.5 Pertanyaan Analisis #5

Sistem rantai pasok dapat dimodelkan sebagai Graf Bipartit (petani di satu sisi, pasar di sisi lain, distributor sebagai penghubung). Jelaskan bagaimana membagi node ke dua himpunan menggunakan BFS-based 2-coloring. Jika graf TIDAK bipartit (ada edge petani-petani atau pasar pasar), apa implikasinya terhadap logistik? Implementasikan fungsi `is_bipartit(graph)` menggunakan BFS berbasis Queue Linked List dan analisis Big-O-nya.

Jawaban :

Sistem rantai pasok dapat dimodelkan sebagai graf bipartit, yaitu graf yang node-nya dapat dibagi menjadi dua himpunan sehingga tidak ada edge yang menghubungkan node dalam himpunan yang sama. Pada kasus ini, node seperti petani dapat berada pada satu sisi, sedangkan pasar pada sisi lain, dengan distributor berperan sebagai penghubung.

Pembagian dua himpunan dapat dilakukan menggunakan **BFS-based 2-coloring**. Ide utamanya adalah memberi warna pada node, misalnya warna 0 dan 1. Node awal diberi warna pertama, lalu semua tetangganya diberi warna kebalikan. Proses BFS terus berlanjut hingga seluruh node dikunjungi. Jika ditemukan dua node bertetangga memiliki warna yang sama, maka graf bukan bipartit.

Jika graf **tidak bipartit**, misalnya terdapat edge **petani–petani** atau **pasar–pasar**, maka struktur distribusi menjadi tidak sesuai dengan model rantai pasok ideal. Hal ini dapat menunjukkan jalur distribusi yang tidak realistis, redundansi distribusi, atau potensi ketidakefisienan logistik karena terjadi hubungan antar-entitas pada level yang sama.

Implementasi Fungsi :

```
def is_bipartit(graph):
    warna = {}

    for start in graph.adj:

        if start not in warna:
            q = CircularQueue(len(graph.adj))

            warna[start] = 0
            q.enqueue(start)

            while not q.is_empty():

                u = q.dequeue()

                for edge in graph.tetangga(u):
                    v = edge.dest

                    if v not in warna:
                        warna[v] = 1 - warna[u]
                        q.enqueue(v)

                    elif warna[v] == warna[u]:
                        return False

    return True
```


Kompleksitas waktunya:

- Setiap node dikunjungi satu kali $\rightarrow O(V)$
- Setiap edge diperiksa satu kali $\rightarrow O(E)$

Sehingga total kompleksitas:

$$O(V + E)$$

Kompleksitas ruang juga $O(V)$ karena queue dan struktur warna menyimpan maksimal seluruh node.

BAB VII

KESIMPULAN

7.1 Ringkasan Pencapaian

proyek *Urban Food Supply Chain Management* berhasil membangun sistem distribusi pangan perkotaan yang efisien dengan enam modul utama berbasis Python. Struktur data seperti Graph, Circular Queue, Priority Queue, BST, dan Stack digunakan sesuai kebutuhan sehingga operasi berjalan optimal. Analisis Big-O dan hasil eksperimen menunjukkan kesesuaian antara teori dan praktik, dengan Dijkstra teridentifikasi sebagai komponen paling berat secara komputasi. Sistem juga dilengkapi CLI interaktif yang memudahkan simulasi pengiriman, pencarian rute, dan manajemen stok. Secara keseluruhan, proyek ini membuktikan bahwa pemilihan struktur data yang tepat dapat menghasilkan sistem rantai pasok pangan yang terukur dan efektif.

7.2 Refleksi Trade-off Desain

Dalam pengembangan sistem Urban Food Supply Chain Management,

- Priority Queue berbasis linked list dipilih karena sederhana dan mendukung *dequeue* cepat, namun *enqueue* menjadi lebih lambat.
- BST katalog produk memberi pencarian efisien rata-rata $O(\log n)$, tetapi bisa jatuh ke $O(n)$ jika tidak seimbang.
- Circular Queue menjamin operasi konstan $O(1)$, namun kapasitas tetap membuatnya kurang fleksibel.
- Algoritma Dijkstra manual sesuai aturan proyek, tetapi kompleksitas $O(V^2 + E)$ menjadi beban komputasi utama. Sementara
- Stack untuk log transaksi mendukung undo cepat, namun terbatas pada akses LIFO.

7.3 Saran Pengembangan

- Antrian Pengiriman: Saat ini sistem sudah bisa jalan, tapi kalau data pengiriman makin banyak jadi terasa lambat.

- Katalog Produk: Pencarian produk sudah bagus, tapi kalau jumlah produk bertambah banyak bisa jadi lebih lama
- Gudang (Buffer): Kapasitas gudang sekarang tetap 50 slot. Akan lebih baik kalau bisa ditambah atau dikurangi sesuai kebutuhan, jadi lebih fleksibel.
- Pencarian Rute (Dijkstra): Cara yang dipakai memang benar, tapi kalau jumlah node makin banyak prosesnya jadi berat.
- Log Transaksi: Sudah bisa menyimpan riwayat terbaru dan mendukung undo, tapi kalau mau cari transaksi lama agak susah

LAMPIRAN

A. Pembagian Tugas Anggota

Nama Anggota	Modul yang Dikerjakan
Nigel Efendi.S	Modul 6
Kaysan Nawfal.S	Modul 4 dan 5
M.Farhan M	Modul 1 dan 2
M.Asri Athalah	Modul 3

B. Tabel Runtime Lengkap

BENCHMARK: PRIORITY QUEUE (Antrian Pengiriman)

Analisis Big-O Teoritis:

Operasi	Kompleksitas	Penjelasan
enqueue(pengiriman)	$O(n)$	Traversel linked list cari posisi sisip yang benar
dequeue()	$O(1)$	Ambil head langsung, no traversal
$n \times \text{enqueue}$	$O(n^2)$	Setiap enqueue $O(n)$, dilakukan n kali = total $O(n^2)$

Operasi	S (50)	M (350)	L (1000)
enqueue x n (total)	0.1554 ms	5.6875 ms	37.9837 ms
dequeue x n (total)	0.0084 ms	0.3282 ms	0.5148 ms
enqueue 1 item ke PQ-n ($O(n)$)	0.0037 ms	0.0130 ms	0.0442 ms
dequeue 1 item dari PQ-n ($O(1)$)	0.0004 ms	0.0012 ms	0.0023 ms
350 event campuran (spec)	1.1716 ms	3.4994 ms	8.9878 ms

Kesimpulan:
 "enqueue 1 item ke PQ-n" menunjukkan pertumbuhan linier $O(n)$ - semakin panjang antrian, semakin lama satu insert.
 "dequeue" tetap hampir konstan $O(1)$ di semua ukuran.

lampiran 1 Benchmark-Priority_queue

BENCHMARK: PRIORITY QUEUE (Antrian Pengiriman)

Analisis Big-O Teoritis:

Operasi	Kompleksitas	Penjelasan
enqueue(pengiriman)	$O(n)$	Traversel linked list cari posisi sisip yang benar
dequeue()	$O(1)$	Ambil head langsung, no traversal
$n \times \text{enqueue}$	$O(n^2)$	Setiap enqueue $O(n)$, dilakukan n kali = total $O(n^2)$

Operasi	S (50)	M (350)	L (1000)
enqueue x n (total)	0.1554 ms	5.6875 ms	37.9837 ms
dequeue x n (total)	0.0084 ms	0.3282 ms	0.5148 ms
enqueue 1 item ke PQ-n ($O(n)$)	0.0037 ms	0.0130 ms	0.0442 ms
dequeue 1 item dari PQ-n ($O(1)$)	0.0004 ms	0.0012 ms	0.0023 ms
350 event campuran (spec)	1.1716 ms	3.4994 ms	8.9878 ms

Kesimpulan:
 "enqueue 1 item ke PQ-n" menunjukkan pertumbuhan linier $O(n)$ - semakin panjang antrian, semakin lama satu insert.
 "dequeue" tetap hampir konstan $O(1)$ di semua ukuran.

lampiran 2 Benchmark-Priority_Queue

BENCHMARK: STACK (Log Transaksi LIFO)

Analisis Big-O Teoritis:

Operasi	Kompleksitas	Penjelasan
push(data)	$O(1)$	Buat node baru, update top = node baru
pop()	$O(1)$	Ambil top.data, geser top = top.next
is_empty()	$O(1)$	Cek size == 0
$n \times \text{push}$	$O(n)$	Loop linear, bukan kompleksitas push-nya

Operasi	S (50)	M (350)	L (1000)
push x n (total)	0.0176 ms	0.0903 ms	0.3079 ms
pop x n (total)	0.0241 ms	0.0737 ms	0.2168 ms
push 1 item (isolasi $O(1)$)	0.0005 ms	0.0004 ms	0.0004 ms
pop 1 item (isolasi $O(1)$)	0.0004 ms	0.0004 ms	0.0005 ms
350 event campuran (spec)	0.0776 ms	0.0687 ms	0.0665 ms

Kesimpulan:
 push dan pop terisolasi menunjukkan waktu hampir konstan di semua n, mengkonfirmasi kompleksitas $O(1)$ untuk kedua operasi Stack.

lampiran 3 Benchmark-Stack

BENCHMARK: BST KATALOG PRODUK

Analisis Big-O Teoritis:

Operasi	Kompleksitas	Penjelasan
insert(produk)	$O(\log n)$	Berdasarkan kode di tiap level, traversal atas-bawah
search(kode)	$O(\log n)$	BST property: tiap level eliminasi setengah subtree
update_stok(kode, delta)	$O(\log n)$	Search $O(\log n)$ + update $O(1)$ = $O(\log n)$
filter_kadaluarsa(hari)	$O(n)$	Inorder traversal kunjungi semua node
inorder()	$O(n)$	Kunjungi setiap node: kiri -> root -> kanan

Operasi	S (50)	M (350)	L (1000)
insert x n (acak, total)	0.0092 ms	0.3638 ms	1.6563 ms
insert 1 item ke BST-n ($O(\log n)$)	0.0044 ms	0.0017 ms	0.0018 ms
search 1 item di BST-n ($O(\log n)$)	0.0027 ms	0.0022 ms	0.0046 ms
update_stok ($O(\log n)$)	0.0002 ms	0.0015 ms	0.0053 ms
filter_kadaluarsa(7) ($O(n)$)	0.0000 ms	0.0004 ms	0.1661 ms
inorder() traversal ($O(n)$)	0.0002 ms	0.0735 ms	0.1566 ms

Kesimpulan:
 insert/search/update menunjukkan pertumbuhan sub-linier $O(\log n)$: dari $n=50$ ke $n=1000$ (20x lipat), waktu hanya naik ~4x (log ratio).
 filter/inorder menunjukkan pertumbuhan linier sesuai $O(n)$.

lampiran 4 Benchmark-BST_Katalog

BENCHMARK: GRAPH RANTAI PASOK (Adjacency List)

Analisis Big-O Teoritis:

Operasi	Kompleksitas	Penjelasan
tambah_node(id, tipe)	$O(1)$	Dict insert + init None pointer
tambah_jalur(u, v, j, b)	$O(1)$	Prepend Edgemode di linked list u dan v
tetangga(u)	$O(\deg)$	Traversel linked list milik u, deg = jumlah tetangga u
BFS mutit_kompleksitas	$O(V+E)$	Kunjungi setiap node dan edge tepat sekali
BFS mutit_ofs	$O(V+E)$	Rekursif, kunjungi setiap node dan edge tepat sekali

Operasi	S (50)	M (350)	L (1000)
tambah_node x n (total)	0.0208 ms	0.1455 ms	0.3897 ms
tambah_jalur x n (total)	0.0355 ms	0.1708 ms	0.7621 ms
tetangga(u) deg=1 ($O(\deg)$)	0.0005 ms	0.0010 ms	0.0002 ms
tetangga(u) deg=20 ($O(\deg)$)	0.0010 ms	0.0017 ms	0.0000 ms
BFS mutit_kompleksitas ($O(V+E)$)	0.0050 ms	0.1277 ms	0.1293 ms

Kesimpulan:
 tambah_node/tambah_jalur mendekati $O(1)$ - hampir tidak berubah antar ukuran. tetangga(deg=20) ~ 20x lebih lambat dari deg=1, mengkonfirmasi $O(\deg)$. BFS naik linier sesuai $O(V+E)$.

lampiran 5 Benchmark-Graph

BENCHMARK: DIJKSTRA BIAYA MINIMAL

Analisis Big-O Teoritis:

Operasi	Kompleksitas	Penjelasan
diijkstra_biaya(graph, asal)	$O(V^2+E)$	V iterasi * $O(V)$ scan minimum + $O(E)$ relaksasi total
rekonstruksi_jalur(parent, s, t)	$O(V)$	Traversel parent[] dari tujuan ke asal, mm V langkah
diijkstra + rekonstruksi (combined)	$O(V^2+E)$	Dibutuhkan diijkstra, rekonstruksi $O(V)$ bersifat minor

Operasi	S (50)	M (350)	L (1000)
diijkstra_biaya ($O(V^2+E)$)	0.1267 ms	18.2959 ms	58.7992 ms
rekonstruksi_jalur ($O(V)$)	0.0018 ms	0.0000 ms	0.0015 ms
diijkstra + rekonstruksi (coba)	0.2435 ms	8.6300 ms	62.5370 ms

Perbandingan	Rasio Waktu	Rasio V^2	Kesimpulan
S (50) vs M (350)	11.52x	49.00x	~ 50% dari 49% dev
M (350) vs L (1000)	5.71x	8.16x	~ 70% dari 56% dev

Kesimpulan:
 Dijkstra menunjukkan pertumbuhan mendekati $O(V^2)$: dari $V=50$ ke $V=1000$ (20x lebih besar), waktu naik ~48x. rekonstruksi_jalur jauh lebih cepat karena $O(V)$ linier, sesuai teori.

lampiran 6 Benchmark-Dijkstra

C. Log AI

```
=====
AI_LOG - Sesi Pengerjaan Tugas Besar ASD
Mata Kuliah : ELT60213 - Algoritma dan Struktur Data
Topik       : 10 - Urban Food Supply Chain Management
Tanggal Sesi : 15 Mei 2026
AI Assistant : Claude (Anthropic) - claude.ai
=====

-----

[001] PEMBUKAAN PROYEK
-----

Prompter:
    hai jadikan aku ada tugas besar asd dari dosen ku, kita disuruh
    membuat sistem urban food supply chain management seperti ini
    [melampirkan gambar slide panduan dosen Topik 10]

Jawaban AI:
    Menampilkan pertanyaan klarifikasi:
    - Bahasa pemrograman yang digunakan → Python
    - Apa yang dibutuhkan sekarang → semua format sudah disediakan dosen,
      mulai dari struktur folder sampai starter code
    - Modul mana yang dikerjakan pertama → akan mengirimkan starter code

-----

[002] MELENGKAPI STARTER CODE
-----

Prompter:
    [melampirkan starter.py dari dosen]
    ini starter code yang perlu kamu lengkapi, lengkapi lah bagian todo
    dan jangan menggunakan library bawaan python pada bagian struktur data

Jawaban AI:
    Melengkapi seluruh bagian TODO pada starter code:
    1. CircularQueue.enqueue() - pointer rear melingkar dengan modulo
    2. CircularQueue.dequeue() - pointer front melingkar, bersihkan slot
    3. PriorityQueueKirim.enqueue() - insertion terurut linked list O(n)
    4. PriorityQueueKirim.dequeue() - ambil head O(1)
    5. Stack.push() - sisip di depan linked list O(1)
    6. Stack.pop() - ambil top O(1)
    7. BSTKatalog.insert() - rekursif, kunci kode_produk
    8. BSTKatalog.search() - rekursif O(log n)
    9. BSTKatalog.update_stok() - search + update in-place
    10. BSTKatalog.filter_kadaluarsa() - inorder traversal + filter
    11. BSTKatalog.inorder() - left → root → right
    12. GraphRantaiPasok.tambah_node() - dict insert O(1)
    13. GraphRantaiPasok.tambah_jalur() - prepend linked list dua arah
    14. GraphRantaiPasok.tetangga() - traversal linked list
    15. dijkstra_biaya() - O(V²+E) tanpa heapq
    16. rekonstruksi_jalur() - traversal parent array O(V)
    17. Implementasi CLI loop lengkap: KIRIM, PROSES_KIRIM, RUTE_MURAH,
        CEK_STOK, KADALUARSA, LAPORAN_DISTRIBUSI, BUFFER, BANTUAN, KELUAR
    Output: file food_supply_chain.py (satu file lengkap)

-----
```

[003] PEMISAHAN KE STRUKTUR FOLDER

Prompter:

[melampirkan screenshot struktur folder proyek]
oke sekarang ini adalah struktur folder utama yang aku buat berdasarkan format dosen ku pada tugas ini. perlu kamu ketahui isi dari data_models adalah mulai dari `import` numpy hingga `@dataclass`. dan untuk generator data juga sesuai yang sudah dibuat dosenku pada starter code. tolong kamu pisahkan dulu perbagian untuk bagian `src/data_structures/`, berdasarkan startercode yang sudah kamu lengkapi sebelumnya dan tambahkan comment sebagai penjelasan

Jawaban AI:

Memisahkan kode menjadi 7 file di `src/data_structures/`:

1. `bst.py` - `BSTNodeProd`, `BSTKatalog` + komentar lengkap
2. `circular_queue.py` - `CircularQueue` + penjelasan circular wrap
3. `priority_queue.py` - `PriorityQueueKirim` + skema prioritas 1/2/3
4. `stack.py` - `Stack` LIFO + penjelasan linked list
5. `graph.py` - `EdgeNode`, `GraphRantaiPasok` + adjacency list
6. `dijkstra.py` - `dijkstra_biaya`, `rekonstruksi_jalur` + pseudocode

[004] PEMISAHAN data_generator.py

Prompter:

sekarang buatlah aku unit test nya untuk tiap data struktur yang sudah kamu pisahkan. oiya aku mau tanya, `generate_data` itu dipakai pas apa? pisahkan sekalian dari starter code ya

Jawaban AI:

Menjawab pertanyaan: `generate_rantai_pasok()` dipanggil SEKALI saat program start di `main()`, menghasilkan "database awal" berisi:

- 26 node (10 Petani + 5 Distributor + 8 Pasar + 3 Gudang)
- ~38 edge jalur distribusi berbobot (deterministik seed=61)
- 12 objek Produk (Beras, Cabai, Tomat, dll.)

Membuat file `src/data_generator.py` (dipisah dari starter code)

Membuat 6 file unit test di `tests/`:

1. `test_bst.py` - 5 class test, ±30 test case
2. `test_circular_queue.py` - 5 class test, ±20 test case (termasuk circular wrap)
3. `test_priority_queue.py` - 5 class test, ±22 test case
4. `test_stack.py` - 4 class test, ±20 test case
5. `test_graph.py` - 5 class test, ±18 test case
6. `test_dijkstra.py` - 5 class test, ±15 test case

[005] PEMBUATAN MODUL 1-3

Prompter:

nah sekarang buatlah aku untuk bagian folder modules pada `src/modules/` ada 1 sampai 6 yaaa

Jawaban AI:

Membuat `src/modules/modul_1.py` - `modul_3.py`:

- `modul_1.py`: `inisialisasi_graph()`, `audit_konektivitas()` (BFS manual), `audit_dfs()`, `tampilkan_info_node()`, `tampilkan_semua_node()`

- modul_2.py: inisialisasi_buffer(), masukkan_stok(), ambil_stok(), cek_buffer_node(), laporan_semua_buffer(), _intip_buffer()
- modul_3.py: tentukan_prioritas(), buat_pengiriman(), proses_pengiriman(), tampilkan_antrian(), ringkasan_antrian()

[006] PEMBUATAN MODUL 4-6 + main.py

Prompter:

lanjutkan pembuatan modulnya, yang sebelumnya belum tuntas

Jawaban AI:

- Membuat src/modules/modul_4.py - modul_6.py + __init__.py + main.py:
- modul_4.py: inisialisasi_katalog(), cek_stok_produk(), perbarui_stok(), daftar_kadaluarsa(), tampilkan_katalog(), tampilkan_produk(), tampilkan_kadaluarsa(), ringkasan_katalog()
 - modul_5.py: cari_rute_termurah(), tampilkan_rute() dengan breakdown biaya per segmen, semua_rute_dari_node(), bandingkan_rute()
 - modul_6.py: jalankan_cli() dengan 9 handler perintah terpisah:
 - _handle_kirim(), _handle_proses_kirim(), _handle_rute_murah(),
 - _handle_cek_stok(), _handle_kadaluarsa(), _handle_laporan(),
 - _handle_buffer(), _handle_audit_jaringan(), _handle_antrian()
 - modules/__init__.py: re-ekspor semua fungsi utama
 - src/main.py: entry point, inisialisasi semua state → jalankan_cli()

[007] BENCHMARK

Prompter:

nahh sekarang buat benchmark nya sesuai dengan format dosenku, (minimal 3 ukuran data berbeda) jadikan semua runtime benchmark dalam satu file aja ya pokoknya sesuai format dosen. tambahkan juga comment seperti analisis big o, dan penjelasan lainnya

Jawaban AI:

- Membuat experiments/benchmark.py dengan:
- 3 ukuran data: S(50), M(350), L(1000) sesuai spesifikasi dosen
 - 7 benchmark terpisah:
 1. benchmark_circular_queue() - 4 skenario: enqueue, dequeue, isolasi, 350 event
 2. benchmark_priority_queue() - 5 skenario: enqueue, dequeue, isolasi, 350 event
 3. benchmark_stack() - 4 skenario: push, pop, isolasi, 350 event
 4. benchmark_bst() - 6 skenario: insert, search, update, filter, inorder
 5. benchmark_graph() - 4 skenario: tambah_node, tambah_jalur, tetangga, BFS
 6. benchmark_dijkstra() - 4 skenario + verifikasi rasio $O(V^2)$
 7. benchmark_mixed_load() - 350 event campuran semua DS (spec dosen)
 - cetak_ringkasan_akhir() - tabel Big-O sesuai tabel 10.3 arsitektur dosen
 - Setiap fungsi dilengkapi docstring analisis Big-O + ekspektasi empiris
 - Cara jalankan: python -m experiments.benchmark

[008] LAPORAN, README, DAN PRESENTASI

Prompter:

[melampirkan PDF panduan lengkap dosen ELT60213]
sekarang tolong aku buat yang belum terbuat, seperti laporan, itu adalah format dari dosen ku. tolong buat laporannya berformat

formal dan jika ada yang tidak bisa kamu masukan berikan keterangan di dalam laporan agar aku bisa menambahkannya seperti screenshot dan lain lain, pastikan semuanya sesuai format dosen. buat juga untuk file readme. buat sekalian pdf untuk presentasinya sesuai format dosen juga

Jawaban AI:

Membuat 3 file:

A. docs/laporan_final.pdf (ReportLab):

- Halaman cover: judul, nama kelompok [perlu diisi], institusi
- Bab I Pendahuluan - latar belakang, tujuan, batasan, param sistem
- Bab II Perancangan - pemilihan DS, justifikasi, pseudocode
- Bab III Implementasi - penjelasan 6 modul, contoh I/O, catatan screenshot
- Bab IV Analisis Big-O - tabel kompleksitas + perbandingan teori vs empiris
- Bab V Hasil Eksperimen - 6 tabel runtime (perlu diisi dari benchmark)
- Bab VI Jawaban 5 Pertanyaan Analisis Wajib
- Bab VII Kesimpulan - pencapaian, tradeoff, saran
- Lampiran - pembagian tugas, log AI

Catatan: semua bagian [SCREENSHOT] dan [DATA] diberi keterangan jelas agar bisa ditambahkan manual.

B. README.md:

- Deskripsi proyek + tabel struktur data & Big-O
- Struktur folder lengkap (tree)
- Cara install, cara jalankan (CLI, test, benchmark)
- Daftar perintah CLI lengkap + contoh sesi
- Parameter sistem & ringkasan Big-O
- Tabel anggota kelompok [perlu diisi]
- Checklist file pengumpulan

C. docs/slide_presentasi.pptx (python-pptx):

Script dibuat, build sempat error pada RGBColor argument (belum selesai di sesi ini)

[009] PERMINTAAN LOG AI

Prompter:

tolong berikan aku untuk isi dari file AI_log/log.promt.txt, pada sesi chat ini. dengan format prompter: dan jawaban ai:

Jawaban AI:

Membuat file AI_Log/Log_prompt.txt berisi ringkasan seluruh percakapan sesi ini dalam format yang diminta.

=====
Total interaksi sesi ini : 9 prompt utama

File yang dihasilkan : 25+ file Python + 1 PDF + 1 README

Baris kode total (approx): ~3.000 baris
=====

=====
SESI 2

AI Assistant : chat GPT
=====

[010] REVISI BENCHMARK SATU FILE

Prompter:

Nah, aku mau itu yang tadi versi revisianmu buat file benchmark ya.
Ingat diganti perdata berbeda. Jadi satu file semuanya.

Jawaban AI:

Merevisi experiments/benchmark.py menjadi satu file benchmark terpadu.

Perubahan utama:

- Memisahkan benchmark berdasarkan ukuran data berbeda:
Small (50), Medium (350), Large (1000)
- Seluruh benchmark dijadikan satu file terintegrasi
- Menambahkan komentar analisis Big-O pada setiap benchmark
- Menambahkan benchmark terpisah untuk:
CircularQueue, PriorityQueue, Stack, BST, Graph, dan Dijkstra
- Menambahkan benchmark mixed workload sesuai spesifikasi dosen
- Menambahkan tabel ringkasan runtime akhir

[011] PEMBELAJARAN GITHUB - PULL REQUEST

Prompter:

Ini sekarang aku enggak tahu caranya pull request gimana.
Ajarin aku ya.

Jawaban AI:

Menjelaskan langkah pull request menggunakan GitHub Desktop:

1. Commit perubahan di branch fitur
2. Push origin branch
3. Klik "Create Pull Request"
4. Membuka halaman GitHub
5. Klik "Compare & Pull Request"
6. Isi title + description PR
7. Klik "Create Pull Request"
8. Klik "Merge Pull Request"
9. Confirm Merge

Menjelaskan bahwa merge PR akan memasukkan isi branch ke branch main.

[012] VERIFIKASI MERGE PULL REQUEST

Prompter:

Pull request successfully merged and closed

Jawaban AI:

Menjelaskan bahwa:

- Pull request sudah berhasil di-merge ke main
- Branch fitur sudah selesai digunakan
- Branch dapat dihapus atau tetap dipakai
- Cara mengecek merge:
 1. Buka branch main
 2. Lihat commit history
 3. Pastikan file hasil revisi sudah muncul

[013] PENJELASAN PUSH ORIGIN

Prompter:

setelah pull request tersetujui itu harus push origin ya?
branchku sekarang sama main udah sama?

Jawaban AI:

Menjelaskan:

- Push origin digunakan untuk mengirim commit lokal ke [GitHub](#)
- Jika PR sudah merged maka isi branch sudah masuk ke main
- Branch fitur dan main menjadi identik setelah merge
- Branch lama tetap bisa dipakai lanjut jika tidak dihapus
- Tidak wajib menghapus branch fitur

[014] PENJELASAN IMPLEMENTASI BST

Prompter:

apakah coding BST yang kamu buat memakai library python?

Jawaban AI:

Menjelaskan bahwa implementasi BST:

- Tidak memakai library struktur data bawaan [Python](#)
- Menggunakan `class` node manual
- Traversal dilakukan rekursif manual
- Sesuai aturan dosen (implementasi sendiri)

Menjelaskan juga:

- search() → traversal node kiri/kanan
- insert() → penempatan node berdasarkan perbandingan kode
- inorder() → traversal left-root-right

[015] PENJELASAN generate_rantai_pasok() DAN seed(61)

Prompter:

terus fungsinya seed(61) dalam startercode dosen ku apa

Jawaban AI:

Menjelaskan bahwa:

- np.random.seed(61) dan random.seed(61)
digunakan untuk membuat hasil random tetap sama setiap dijalankan
- Berguna agar:
 - data benchmark konsisten
 - pengujian unit test stabil
 - hasil seluruh anggota kelompok sama
- generate_rantai_pasok() menghasilkan:
 - node supply chain
 - edge distribusi
 - daftar produk awal

[016] ANALISIS STARTER CODE DOSEN

Prompter:

[mengirim ulang starter code lengkap dosen]

Jawaban AI:

Menganalisis starter code dosen:

- data_models.py berisi dataclass `Produk` dan `Pengiriman`
- generate_rantai_pasok() wajib dipisahkan ke data_generator.py
- `Struktur` data dipisah ke folder data_structures/
- main() hanya sebagai orchestrator CLI
- `Semua` TODO harus diimplementasikan manual tanpa library DS `Python`

[017] PEMISAHAN FILE data_models.py DAN data_generator.py

Prompter:

implementasikan juga data_models ku

Jawaban AI:

Membantu memisahkan:

- src/data_models.py
- src/data_generator.py

Isi data_models.py:

- `import` numpy/random
- `seed(61)`
- konstanta kategori produk
- dataclass `Produk`
- dataclass `Pengiriman`

Isi data_generator.py:

- generate_rantai_pasok(seed=61)
- generate node
- generate edge acak deterministik
- generate produk awal

[018] PENJELASAN TYPE HINT BST

Prompter:

loh itu yang bst langsung parameternya langsung produk emang bisa?
bukanya produk : `Produk`

Jawaban AI:

Menjelaskan bahwa:

- produk: `Produk` adalah type hint
- `Type` hint hanya penjas tipe data
- `Tetap` dipanggil menggunakan objek `Produk` biasa
- `Contoh`:
 `bst.insert(produk_obj)`
- `Tidak` mempengaruhi runtime `Python`

[019] PEMBUATAN COMMIT MESSAGE

Prompter:

ini kan aku mau commit aku membuat file data generator + isi syntax nya
summary nya diisi apa dan deskripsinya gimana

Jawaban AI:

Memberikan contoh commit message sesuai format dosen:

- feat(generator): tambah data generator rantai pasok
- feat(data_models): implementasi dataclass produk dan pengiriman
- feat(test): tambah unit test BST katalog produk

Menjelaskan:

- feat = fitur baru
- fix = perbaikan bug
- refactor = restrukturisasi kode
- exp = eksperimen benchmark

[020] ANALISIS UNIT TEST BST

Prompter:

[mengirim file test_bst.py lengkap]

Jawaban AI:

Menganalisis unit test BST:

- Struktur test sudah benar
- Fixture bst_terisi valid
- Test mencakup:
 - insert, search, update_stok,
 - filter_kadaluarsa, inorder, edge case

Menjelaskan kemungkinan error:

- pytest belum terinstall
- import path belum sesuai
- interpreter Python VS Code berbeda

[021] INSTALL pytest DAN KONFIGURASI VS CODE

Prompter:

Import "pytest" could not be resolved

Jawaban AI:

Membimbing:

1. Install pytest menggunakan pip
2. Pilih interpreter Python yang benar di VS Code
3. Reload Window
4. Jalankan:

python -m pytest tests/test_bst.py -v

Menjelaskan bahwa:

- Error muncul karena VS Code memakai interpreter berbeda
- pytest terinstall di Python 3.14 tetapi terminal memakai msys64

[022] PERBAIKAN COMMAND pytest DI POWERSHELL

Prompter:

Unexpected token '-m' in expression or statement

Jawaban AI:

Menjelaskan syntax PowerShell yang benar:

```
& "C:\path\python.exe" -m pytest tests/test_bst.py -v
```

Menjelaskan:

- operator & wajib dipakai di PowerShell
- path Python harus dibungkus tanda kutip

[023] ERROR MODULE 'src'

Prompter:

ModuleNotFoundError: No module named 'src'

Jawaban AI:

Menjelaskan penyebab:

- File modul dijalankan langsung
- Python tidak menganggap root project sebagai package root

Solusi:

- Jalankan dari root project:
python -m src.modules.modul_4
- Bukan:
python src/modules/modul_4.py

[024] ERROR numpy TIDAK TERDETEKSI

Prompter:

ModuleNotFoundError: No module named 'numpy'

Jawaban AI:

Menjelaskan bahwa:

- numpy terinstall di interpreter berbeda
- VS Code/terminal masih memakai Python msys64

Solusi:

- gunakan interpreter:
Python 3.14.5
- install numpy pada interpreter yang sama
- cek:
python -m pip show numpy

[025] ANALISIS MODUL TIDAK BISA DIRUN LANGSUNG

Prompter:

coba kamu lihat codinganku itu,
apa memang kalau di file modul gabisa dijalankan

Jawaban AI:

Menjelaskan bahwa:

- modul_4.py memang bukan entry point utama
- modul hanya berisi fungsi helper
- Tidak ada:

```
if __name__ == '__main__'
```

Menjelaskan arsitektur proyek:

- modul_1-6 → helper layer
- main.py → orchestration layer
- CLI baru bisa berjalan jika seluruh modul terhubung di main.py

[026] ANALISIS ARSITEKTUR PIPELINE DOSEN

Prompter:

[mengirim tabel 10.3 Arsitektur Pipeline]

Jawaban AI:

Menganalisis bahwa:

- Sistem baru benar-benar runnable setelah modul 1-6 selesai
- Semua struktur data saling bergantung:
Graph → Queue → Priority Queue → BST → Dijkstra → CLI
- main.py menjadi integrator seluruh modul
- Tabel dosen dipakai sebagai acuan pipeline sistem penuh

[027] PENJELASAN requirements.txt

Prompter:

file requirement itu yang benar tulisannya gimana

Jawaban AI:

Menjelaskan:

- Penulisan standar Python adalah:
requirements.txt
- Jika dosen menulis:
requirments.txt
maka kemungkinan typo

Saran:

- Ikuti nama dari dosen jika wajib sesuai template
- Tetapi standar industri tetap requirements.txt

[028] PERMINTAAN AI LOG LANJUTAN

Prompter:

nah sekarang aku minta ai log pada sesi chat ini,
kamu tinggal buat lanjutannya dari yang aku kirim

Jawaban AI:

Membuat lanjutan AI_LOG sesuai format sebelumnya:

- nomor berurutan
- format "Prompter" dan "Jawaban AI"
- ringkasan teknis setiap sesi
- mencakup GitHub, BST, pytest, VS Code,
benchmark, struktur modul, dan debugging environment

=====
Total interaksi tambahan sesi ini : 19 prompt utama

Topik utama tambahan :

- [GitHub Pull Request](#)
- VS [Code](#) & pytest debugging
- [Struktur](#) modular [Python](#)
- [Environment](#) interpreter
- [Benchmark](#) revisi
- [Dokumentasi](#) AI [Log](#)

=====